ادامه SVM

The optimal margin classier

MMC

- we will assume that we are given a training set that is linearly separable
- How will we find the one that achieves the **maximum geometric margin**? We can pose the following optimization problem:

$$\max_{\gamma,w,b} \quad \gamma$$
$$\text{s.t.} \quad y^{(i)}\left(w^T x^{(i)} + b\right) \geq \gamma,\ i = 1, \ldots, m$$
$$\| w \| = 1$$

  - we want to maximize γ, subject to each training example having functional margin at least γ.

The optimal margin classier

- If we could solve the optimization problem above, we'd be done. But the "|w|=1" constraint is a nasty (non-convex) one, and this problem certainly isn't in any format that we can plug into standard optimization software to solve.
- So, let's try transforming the problem:

functional margin

$$\max_{\hat{\gamma},w,b} \quad \frac{\hat{\gamma}}{\| w \|}$$

$$\text{s.t.} \quad y^{(i)}\left(w^T x^{(i)} + b\right) \geq \hat{\gamma}, \; i = 1, \ldots, m$$

  - Here, we're going to maximize $\hat{\gamma}/\| w \|$, subject to the functional margins all being at least $\hat{\gamma}$.
  - we've gotten rid of the constraint $\| w \| = 1$ that we didn't like. The downside is that we now have a nasty (again, non-convex) objective $\frac{\hat{\gamma}}{\|w\|}$ function.

The optimal margin classier

$w^T x + b = 0$

$2w^T x + 2b = 0$

- Recall our earlier discussion that we can add an arbitrary scaling constraint on w and b without changing anything.

- We will introduce the scaling constraint that the functional margin of w, b with respect to the training set must be 1 : $\hat{\gamma} = 1.$

- Plugging this into our problem above, and noting that maximizing $\hat{\gamma}/\| w \| = 1/\| w \|$ is the same thing as minimizing $\| w \|^2$, we now have the following optimization problem:

$$\min_{w,b} \quad \frac{1}{2} \| w \|^2$$

$$\text{s.t.} \quad y^{(i)}\left(w^T x^{(i)} + b\right) \geq 1, \; i = 1, \ldots, m$$

The above is an optimization problem with a **convex quadratic objective** and only *linear constraints*.

The optimal margin classier

$$\min_{w,b} \quad \frac{1}{2} \| w \|^2$$
$$\text{s.t.} \quad y^{(i)}\left(w^T x^{(i)} + b\right) \geq 1, \; i = 1, \ldots, m$$

- Its solution gives us the **optimal margin classifier**. This optimization problem can be solved using commercial quadratic programming (QP).
- What we will still do is to make a digression to talk about **Lagrange duality**.
- This will lead us to our optimization problem's dual form, which will play a key role in allowing us to use **kernels** to get optimal margin classifiers to work efficiently in very high dimensional spaces.
- The dual form will also allow us to derive an efficient algorithm for solving the above optimization problem that will typically do much better than generic QP software.

Lagrange duality

- Consider a problem of the following form:

$$\begin{aligned} \min_{w} \quad & f(w) \\ \text{s.t.} \quad & h_i(w) = 0,\ i = 1, \ldots, l \end{aligned}$$

- define the Lagrangian to be:

$$\mathcal{L}(w, \beta) = f(w) + \sum_{i=1}^{l} \beta_i h_i(w)$$

- Here, the β_i 's are called the Lagrange multipliers. We would then find and set $\mathcal{L}$'s partial derivatives to zero:

$$\frac{\partial \mathcal{L}}{\partial w_i} = 0; \quad \frac{\partial \mathcal{L}}{\partial \beta_i} = 0,$$

and solve for w and β.

Lagrange duality

Consider the following, which we'll call the primal optimization problem:

$$\begin{aligned} \min_{w} \quad & f(w) \\ \text{s.t.} \quad & g_i(w) \leq 0, \; i = 1, \ldots, k \\ & h_i(w) = 0, \; i = 1, \ldots, l. \end{aligned}$$

To solve it, we start by defining the generalized Lagrangian

$$\mathcal{L}(w, \alpha, \beta) = f(w) + \sum_{i=1}^{k} \alpha_i g_i(w) + \sum_{i=1}^{l} \beta_i h_i(w)$$

Here, the α_i 's and β_i 's are the Lagrange multipliers.

Lagrange duality

Consider the quantity

$$\theta_{\mathcal{P}}(w) = \max_{\alpha,\beta:\alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta).$$

Here, the " $\mathcal{P}$ " subscript stands for "primal." Let some w be given. If w violates any of the primal constraints (i.e., if either $g_i(w) > 0$ or $h_i(w) \neq 0$ for some i, then you should be able to verify that

$$\theta_{\mathcal{P}}(w) = \begin{cases} f(w) & \text{if } w \text{ satisfies primal constraints} \\ \infty & \text{otherwise.} \end{cases}$$

Lagrange duality

$$\min_{w} \theta_{\mathcal{P}}(w) = \min_{w} \max_{\alpha,\beta : \alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta),$$

مسئله اصلی

$$d^* = \max_{\alpha,\beta : \alpha_i \geq 0} \min_{w} \mathcal{L}(w, \alpha, \beta) \leq \min_{w} \max_{\alpha,\beta : \alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta) = p^*.$$

حل مسئله dual

مسئله dual

$d^* = p^*$ ← (KKT) شرایط تحت و Convex باشد $\mathcal{L}$ اگر مسئله

Optimal margin classifiers

$$\min_{w,b} \quad \frac{1}{2} \| w \|^2$$
$$\text{s.t.} \quad y^{(i)}\left(w^T x^{(i)} + b\right) \geq 1, \; i = 1, \ldots, m$$

When we construct the Lagrangian for our optimization problem we have:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \| w \|^2 - \sum_{i=1}^{m} \alpha_i \left[y^{(i)}\left(w^T x^{(i)} + b\right) - 1 \right] \qquad (21)$$

Let's find the dual form of the problem. which we'll do by setting the derivatives of $\mathcal{L}$ with respect to w and b to zero. We have:

$$\nabla_w \mathcal{L}(w, b, \alpha) = w - \sum_{i=1}^{m} \alpha_i y^{(i)} x^{(i)} = 0$$

$$w = \sum_{i=1}^{m} \alpha_i y^{(i)} x^{(i)} \qquad (22)$$

$$\frac{\partial}{\partial b} \mathcal{L}(w, b, \alpha) = \sum_{i=1}^{m} \alpha_i y^{(i)} = 0 \qquad (23)$$

Optimal margin classifiers

If we take the definition of w in Equation (22) and plug that back into the Lagrangian (Equation 21), and simplify, we get

$$\mathcal{L}(w, b, \alpha) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} y^{(i)} y^{(j)} \alpha_i \alpha_j \left(x^{(i)}\right)^T x^{(j)} - b \sum_{i=1}^{m} \alpha_i y^{(i)}$$

But from Equation (23), the last term must be zero, so we obtain

$$\mathcal{L}(w, b, \alpha) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} y^{(i)} y^{(j)} \alpha_i \alpha_j \left(x^{(i)}\right)^T x^{(j)}$$

$\langle x^{(i)}, x^{(j)} \rangle$

Optimal margin classifiers

Recall that we got to the equation above by minimizing $\mathcal{L}$ with respect to w and b. Putting this together with the constraints $\alpha_i \geq 0$ (that we always had) and the constraint (23), we obtain the following **dual optimization problem**:

$$\max_{\alpha} \quad W(\alpha) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle$$

$$\text{s.t.} \quad \alpha_i \geq 0, \; i = 1, \ldots, m$$

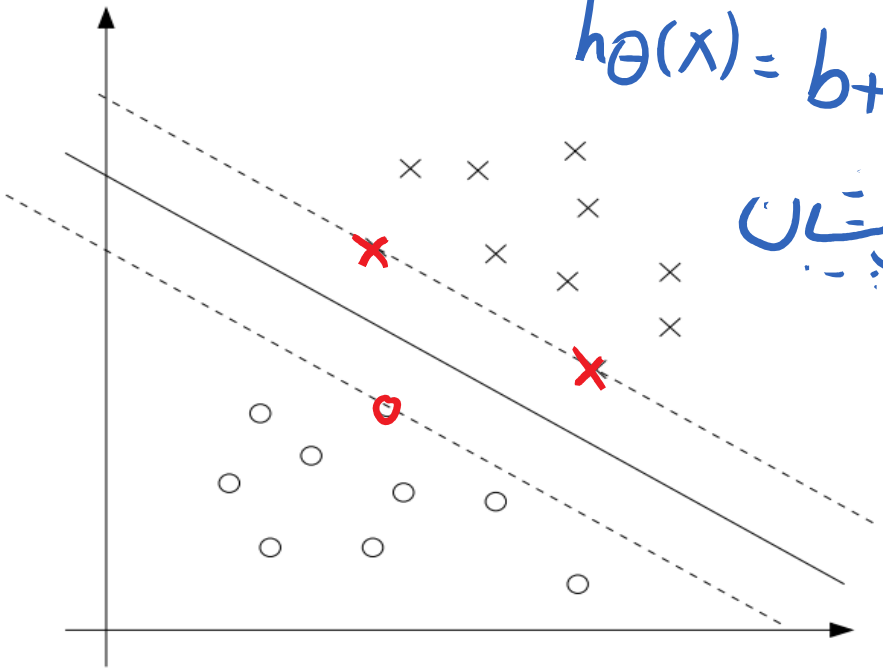
$$\sum_{i=1}^{m} \alpha_i y^{(i)} = 0$$

Optimal margin classifiers

$$w^* = \sum_{i=1}^{m} \alpha_i y^{(i)} x^{(i)}$$

$$b^* = -\frac{\max_{i:y^{(i)}=-1} w^{*T} x^{(i)} + \min_{i:y^{(i)}=1} w^{*T} x^{(i)}}{2}.$$

Optimal margin classifiers



here, these are the three points (one negative and two positive examples)
Thus, only three of the α_i 's - namely, the ones corresponding to these three training examples - will be non-zero at the optimal solution to our optimization problem.
These three points are called the **support vectors** in this problem.

Regularization and the non-separable case

- mapping data to a high dimensional feature space via ϕ does generally increase the likelihood that the data is separable, we can't guarantee that it always will be so.

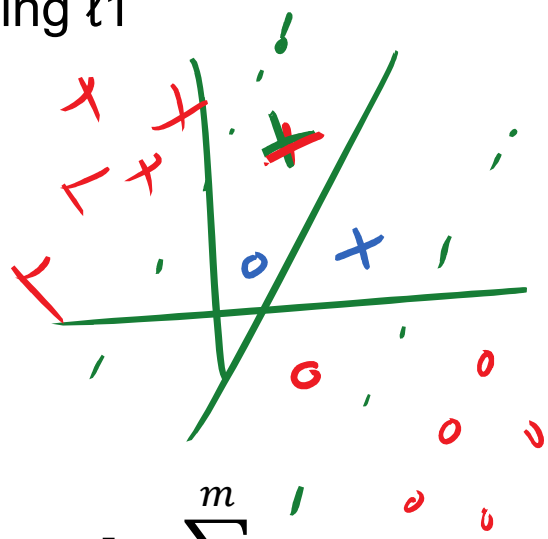
support vector classifier

- To make the algorithm work for non-linearly separable datasets as well as be less sensitive to outliers, we reformulate our optimization (using ℓ1 regularization) as follows:

$$\min_{\gamma,w,b} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m}\xi_i$$

$$\text{s.t.} \quad y^{(i)}\left(w^T x^{(i)} + b\right) \geq 1 - \xi_i, \; i = 1, \ldots, m$$

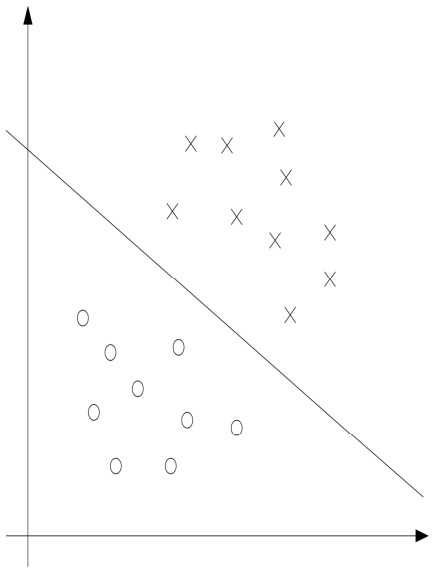
$$\xi_i \geq 0, \; i = 1, \ldots, m$$

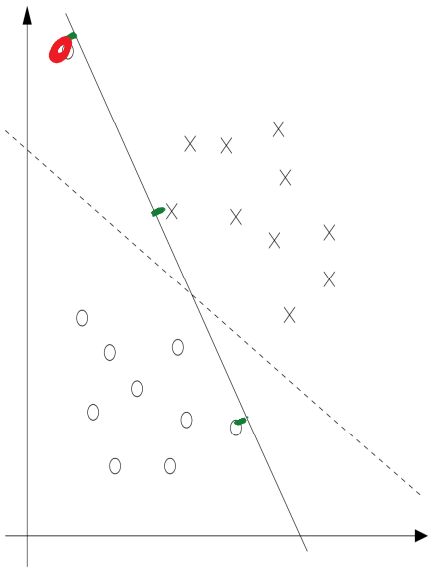


As before, we can form the Lagrangian:

$$\mathcal{L}(w, b, \xi, \alpha, r) = \frac{1}{2}w^T w + C\sum_{i=1}^{m}\xi_i - \sum_{i=1}^{m}\alpha_i\left[y^{(i)}\left(x^T w + b\right) - 1 + \xi_i\right] - \sum_{i=1}^{m} r_i\xi_i$$

Regularization and the non-separable case





Regularization and the non-separable case

Thus, examples are now permitted to have (functional) margin less than 1, and if an example has functional margin $1 - \xi_i$ (with $\xi > 0$), we would pay a cost of the objective function being increased by $C\xi_i$. The parameter C controls the relative weighting between the twin goals of making the $||w||^2$ small (which we saw earlier makes the margin large) and of ensuring that most examples have functional margin at least 1.

Support vector classifier:

$$\max_{w_0, w_1, \dots, w_n, \epsilon_1, \epsilon_2, \dots, \epsilon_m, M} M$$

Subject to

$$\sum_{j=1}^{n} w_j^2 = 1$$

$$y^{(i)}\left(w_0 + w_1 x_1^{(i)} + \dots + w_n x_n^{(i)}\right) \geq M(1 - \epsilon_i)$$

$$\epsilon_i \geq 0 \qquad \sum_{i=1}^{m} \epsilon_i \leq C \qquad C > 0$$

C: هایپرپارامتر

hyperplane, margin

MMC ← C=0

$C \uparrow \rightarrow$ Margin ↑

$C \uparrow \rightarrow$ bias ↑, var ↓

$C \downarrow \rightarrow$ bias ↓, var ↑

$x^{(i)}$ در سمت درست margin است ← $\epsilon_i = 0$

$x^{(i)}$ روی margin است ← $1 > \epsilon_i > 0$

$x^{(i)}$ روی hyperplane است ← $\epsilon_i > 1$

فقط نقاطی که روی حاشیه (مرز) قرار دارند یا از آن تجاوز کرده عبور کرده هستند ← بردارهای پشتیبان (SV) hyperplane

C↑ ⇒ تعداد SV ↑ ، var ↓

C↓ ⇒ ↓ ، bias ↓

overfit
underfit

var↑
bias↑

3- Support vector Machine

SV classifier + Kernel

$X \longrightarrow \phi(x)$

Regularization and the non-separable case

dual form of the problem:

$$\max_{\alpha} \quad W(\alpha) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle$$

$$\text{s.t.} \quad 0 \le \alpha_i \le C, \; i = 1, \ldots, m$$

$$\sum_{i=1}^{m} \alpha_i y^{(i)} = 0$$

$$w = \sum_{i=1}^{m} \alpha_i y^{(i)} x^{(i)}$$

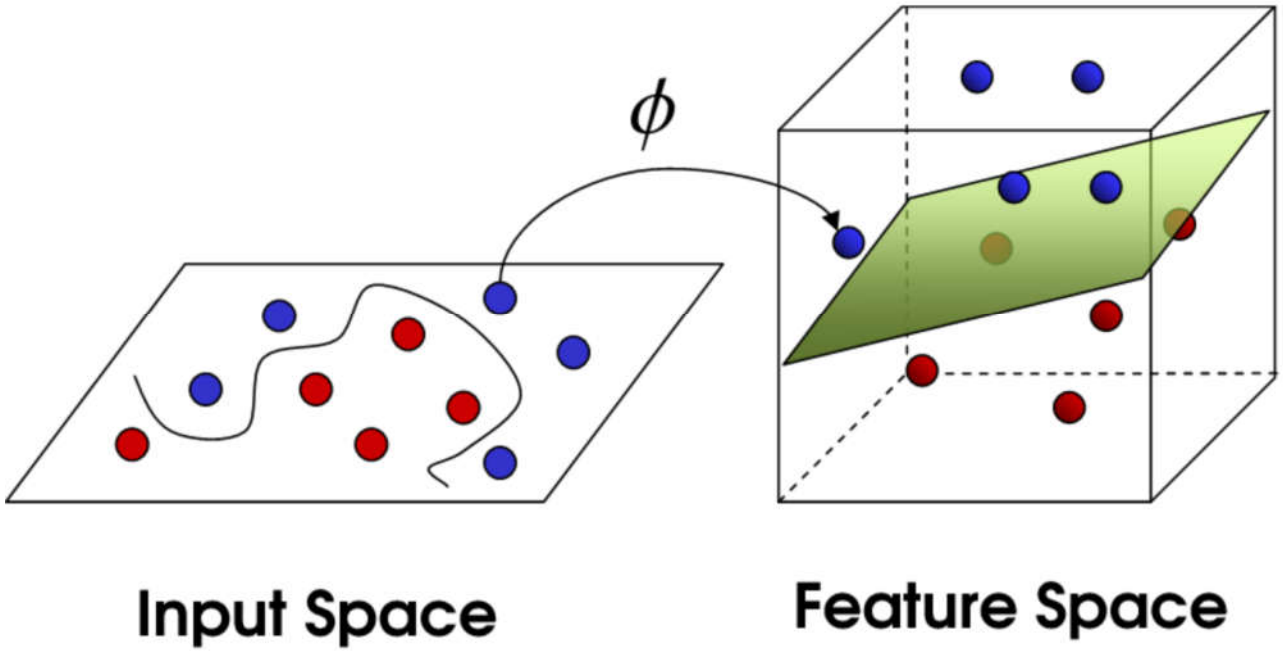
$$h_\theta(x) = w^T x + b = \sum_{i \in S} \alpha_i y^{(i)} \langle x, x^{(i)} \rangle + b \gtrless 0$$

$k(x, x^{(i)})$

(SVM)

SVM (Non-linear decision boundary)

$X \rightarrow \phi(X)$

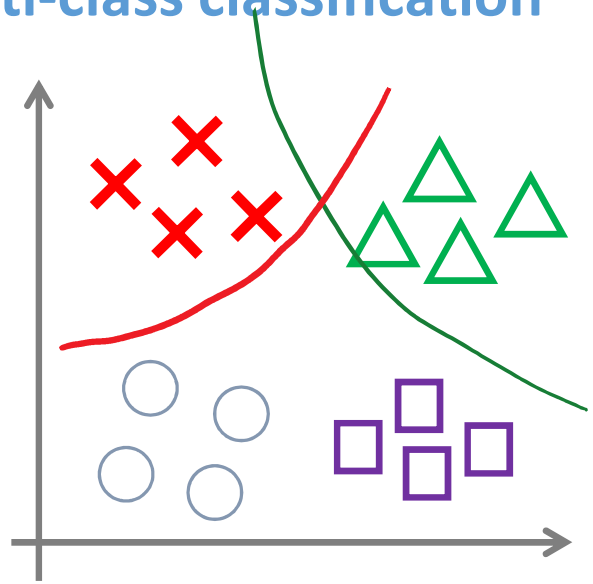


SVM (Non-linear decision boundary)

Common kernel functions include:

- Linear: $\langle x, x' \rangle$
- Polynomial: $(\gamma \langle x, x' \rangle + r)^d$
- rbf: $\exp(-\gamma |x - x'|^2)$ where γ must be greater than 0.
- Sigmoid $(\tanh(\gamma \langle x, x' \rangle + r))$

Multi-class classification



$$y \in \{1, 2, 3, \ldots, K\}$$

$$\max_{k} h_\theta^{(k)}(x_{test}) = y_{test}$$

$$k = 1 \ldots K$$

Many SVM packages already have built-in multi-class classification functionality.

Otherwise, use one-vs.-all method. (Train K SVMs, one to distinguish $y = i$ from the rest, for $i = 1, 2, \ldots, K$), get $\theta^{(1)}, \theta^{(2)}, \ldots, \theta^{(K)}$

Pick class i with largest $(\theta^{(i)})^T x$

Logistic regression vs. SVMs

n =number of features ($x \in \mathbb{R}^{n+1}$), m =number of training examples

- If n is large (relative to m):
  Use logistic regression, or SVM without a kernel (“linear kernel”)

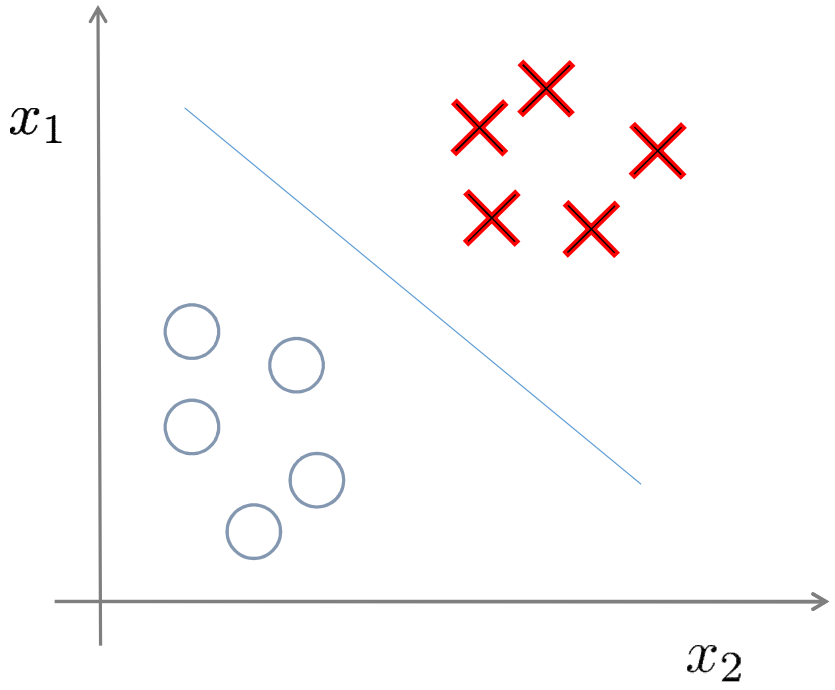
- If n is small, m is intermediate:
  Use SVM with Gaussian kernel

- If n is small, m is large:
  Create/add more features, then use logistic regression or SVM without a kernel

Neural network is likely to work well for most of these settings, but may be slower to train.

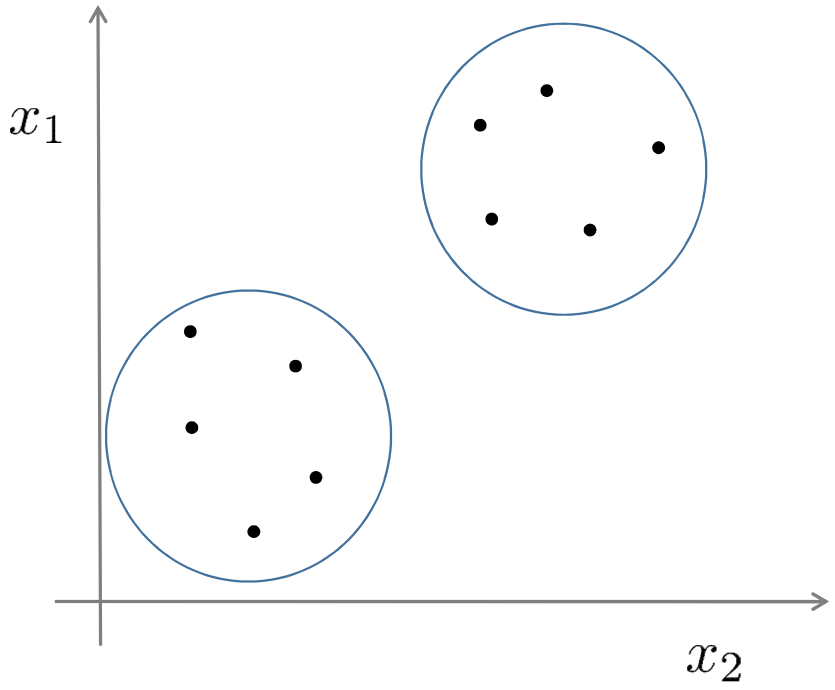
Unsupervised learning introduction

Supervised learning



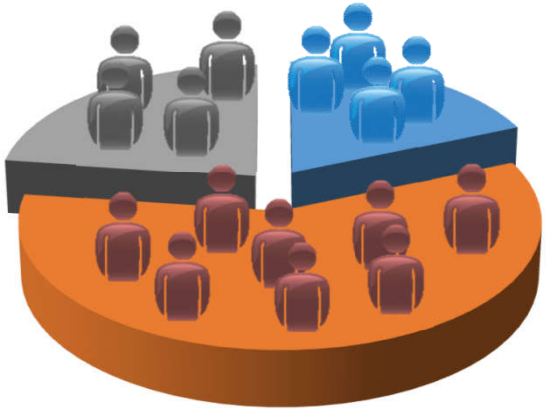
Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), (x^{(3)}, y^{(3)}), \ldots, (x^{(m)}, y^{(m)})\}$

Unsupervised learning

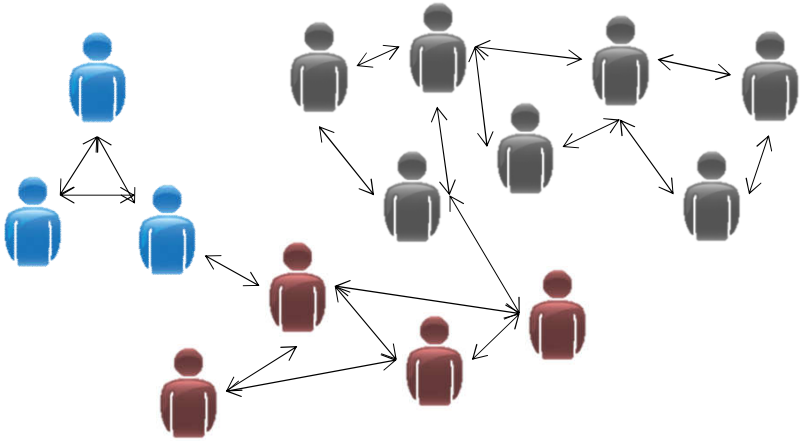


Training set: $\{x^{(1)}, x^{(2)}, x^{(3)}, \ldots, x^{(m)}\}$

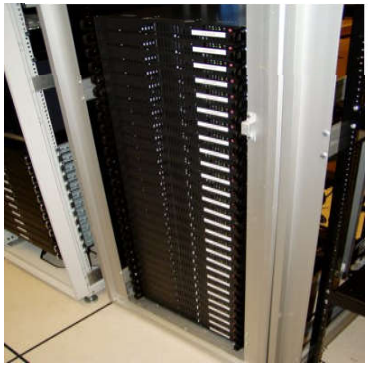
Applications of clustering

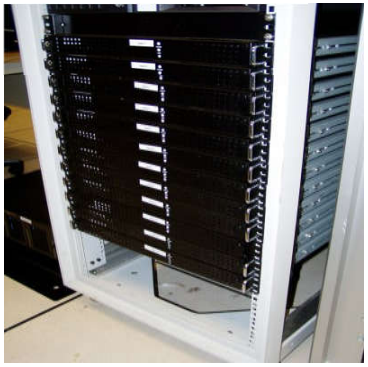


Market segmentation

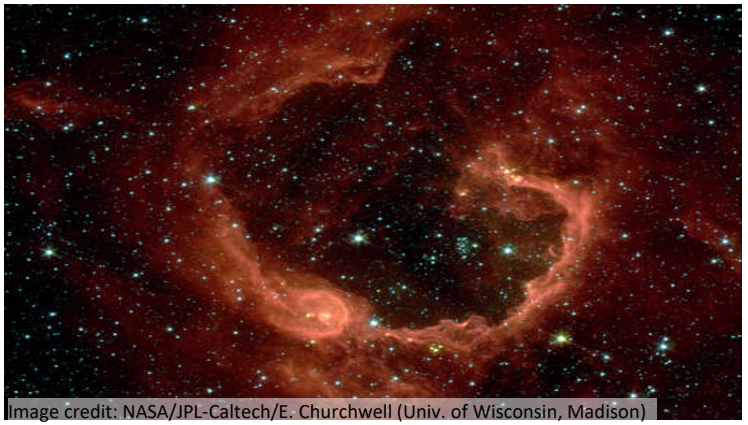


Social network analysis



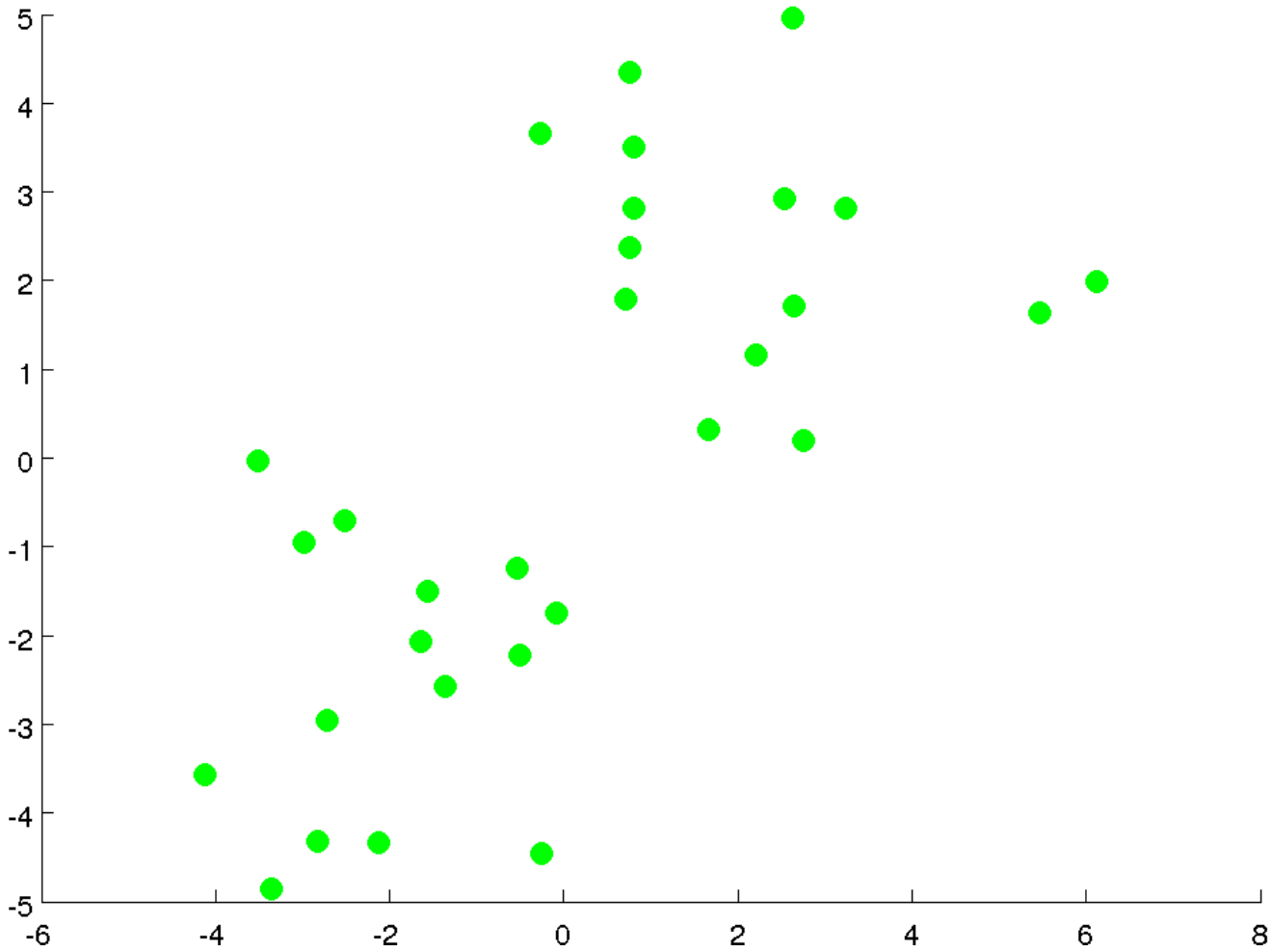


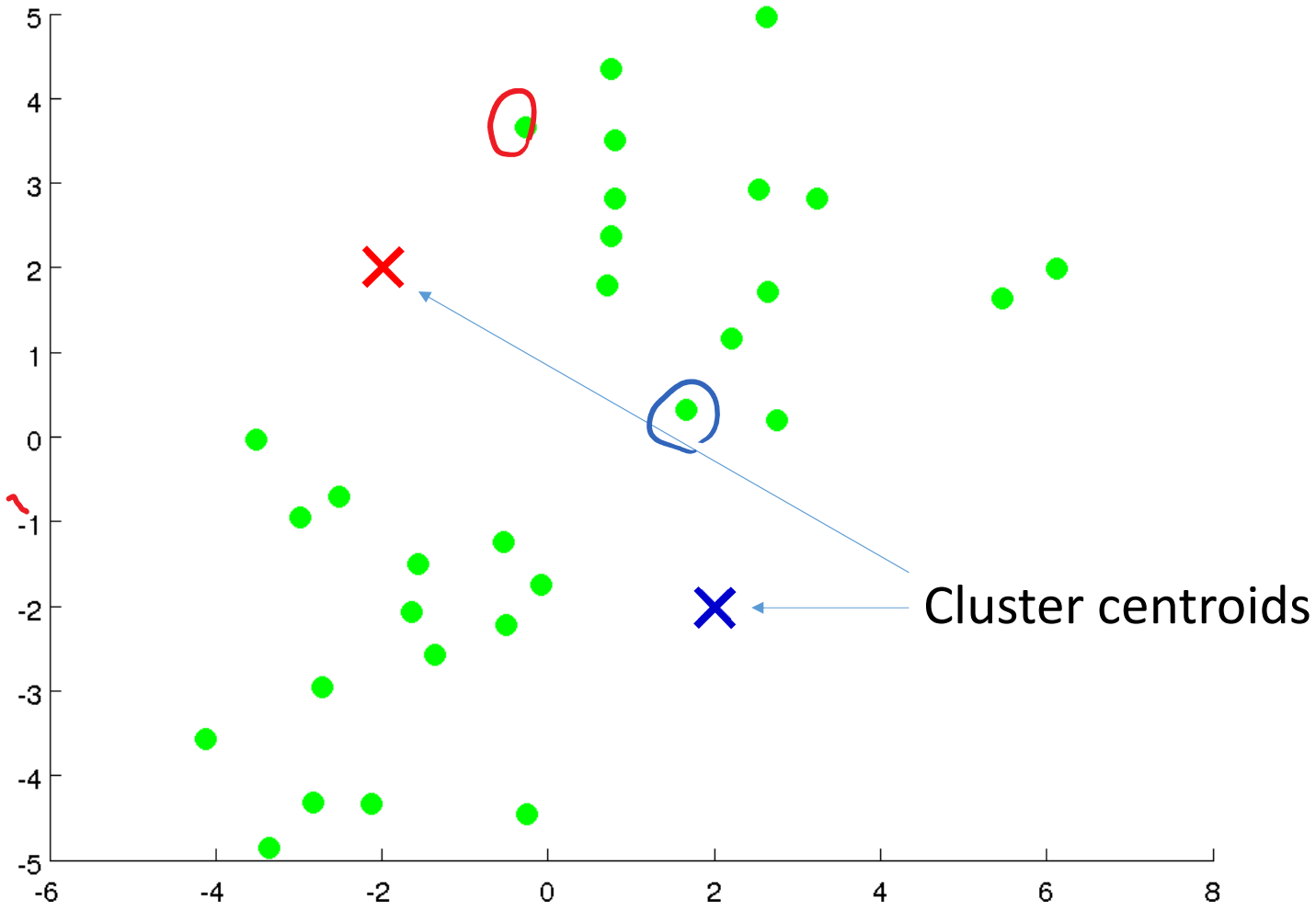
Organize computing clusters

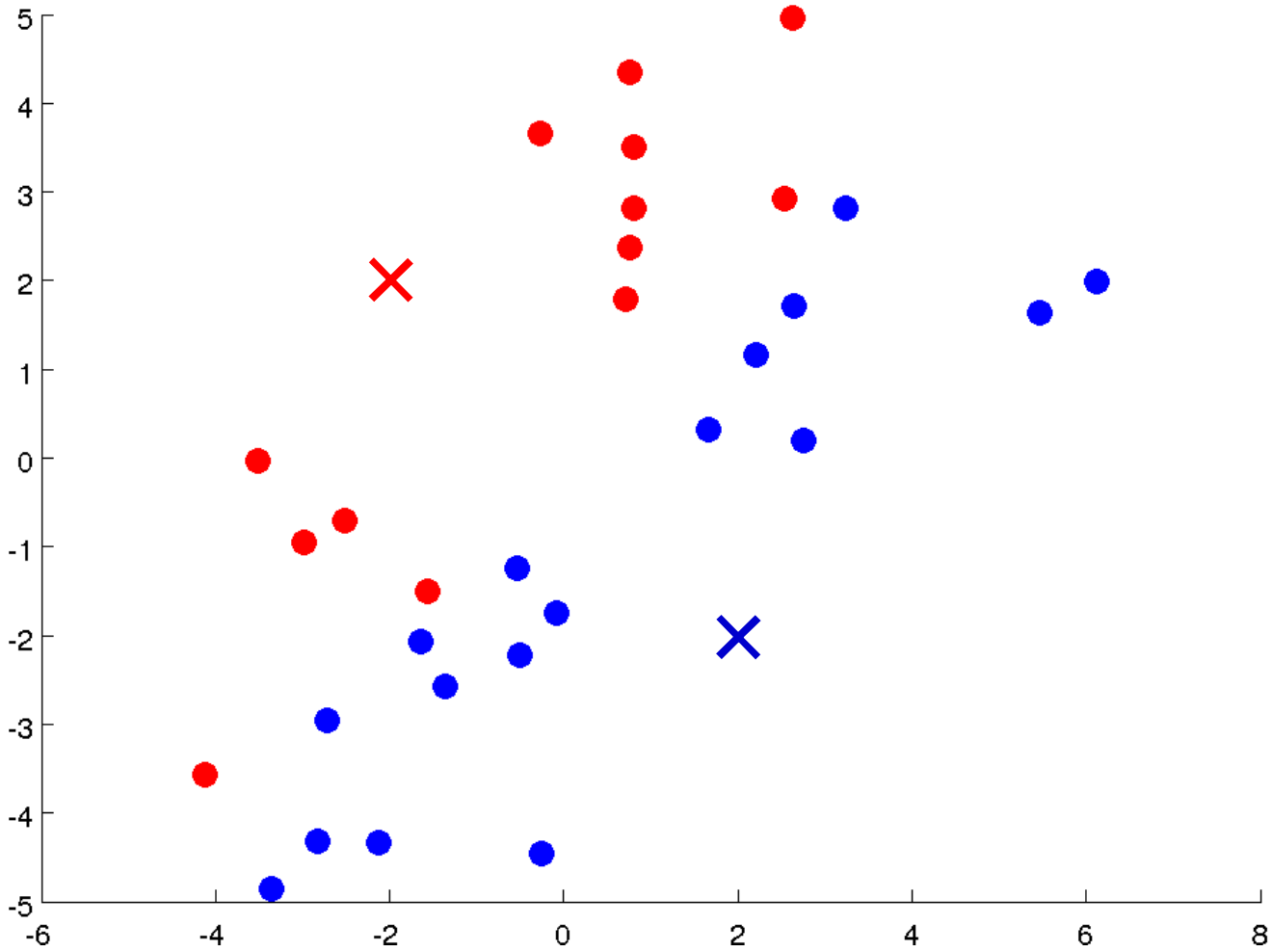


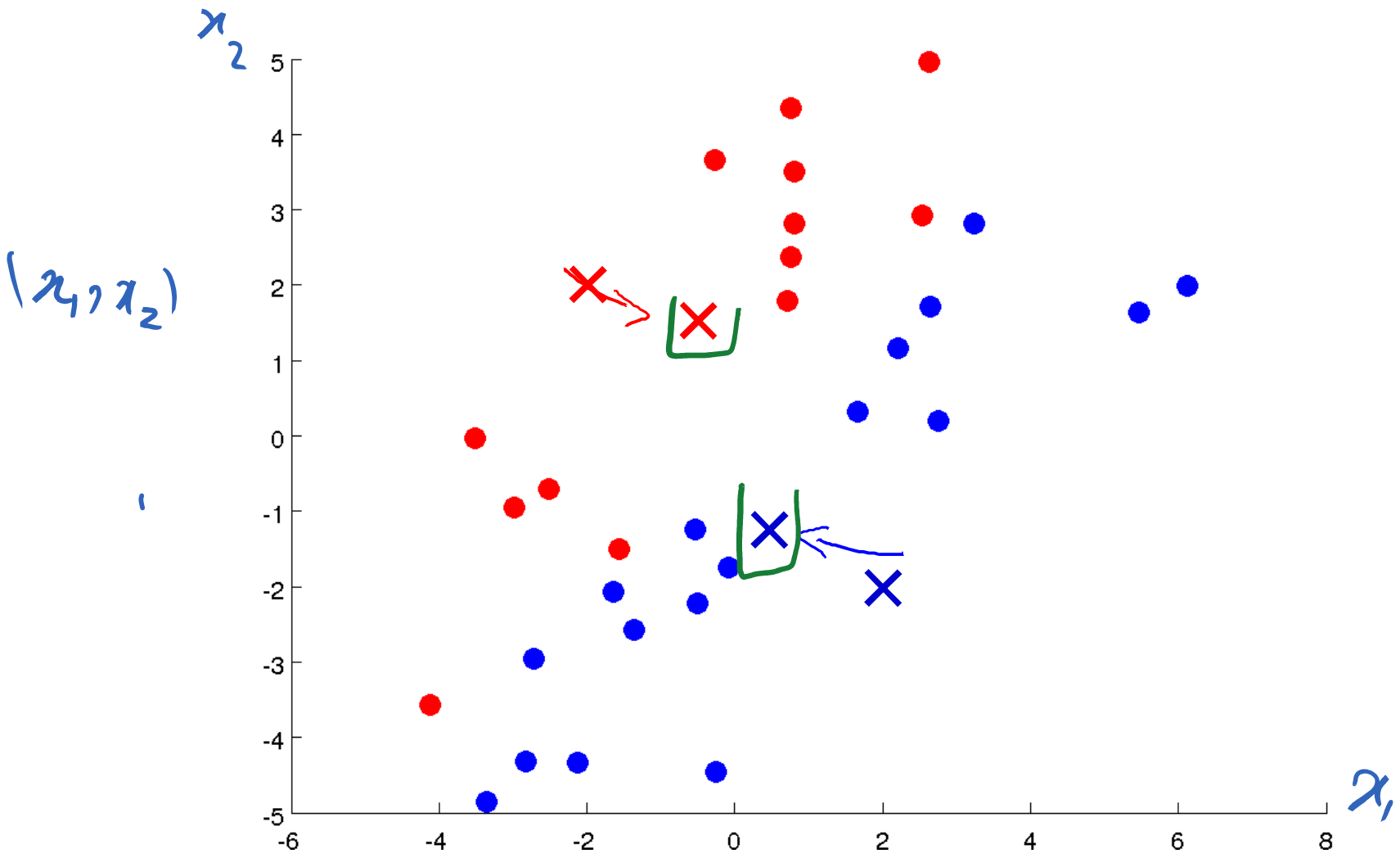
Astronomical data analysis

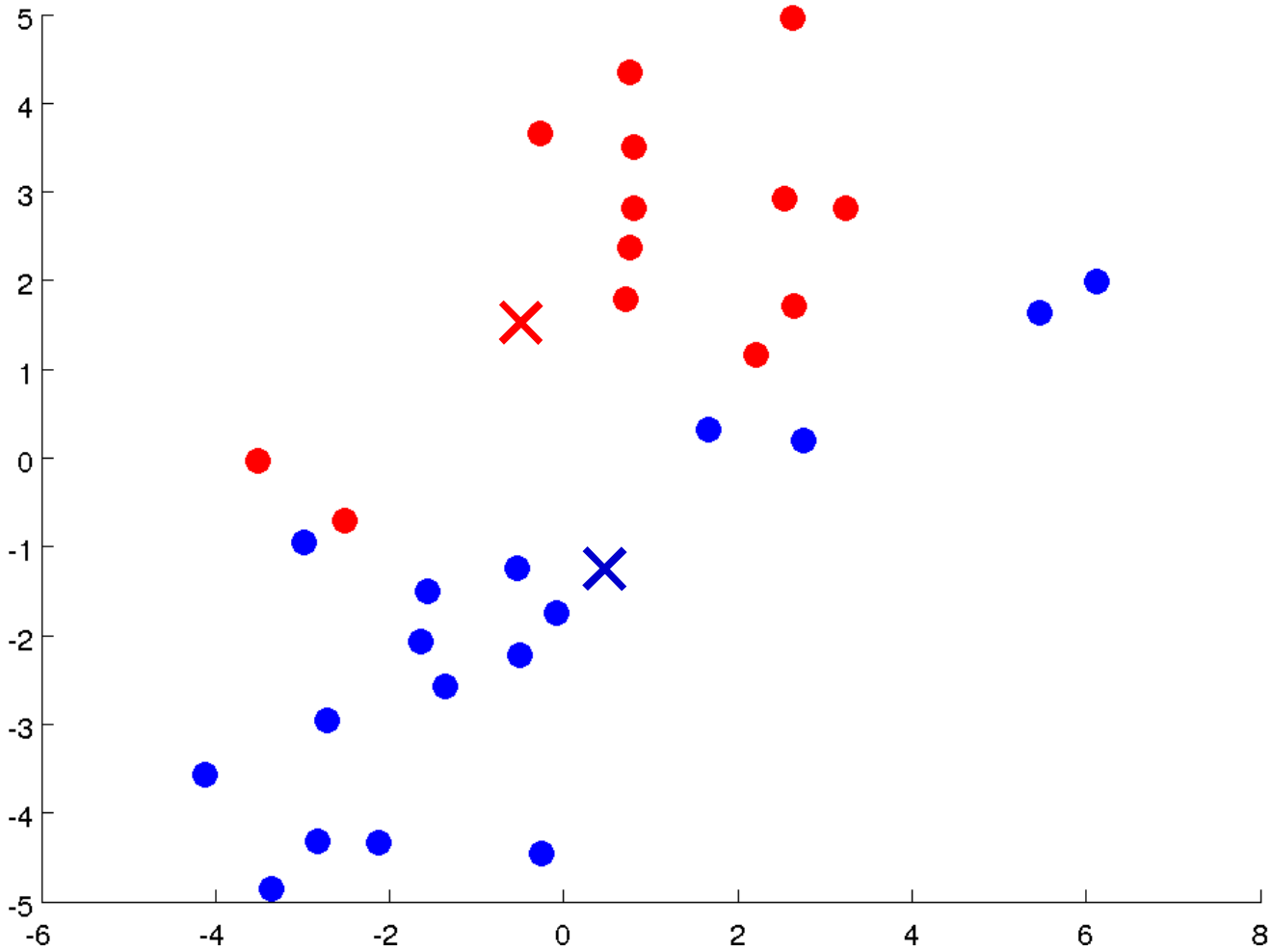
K-means algorithm

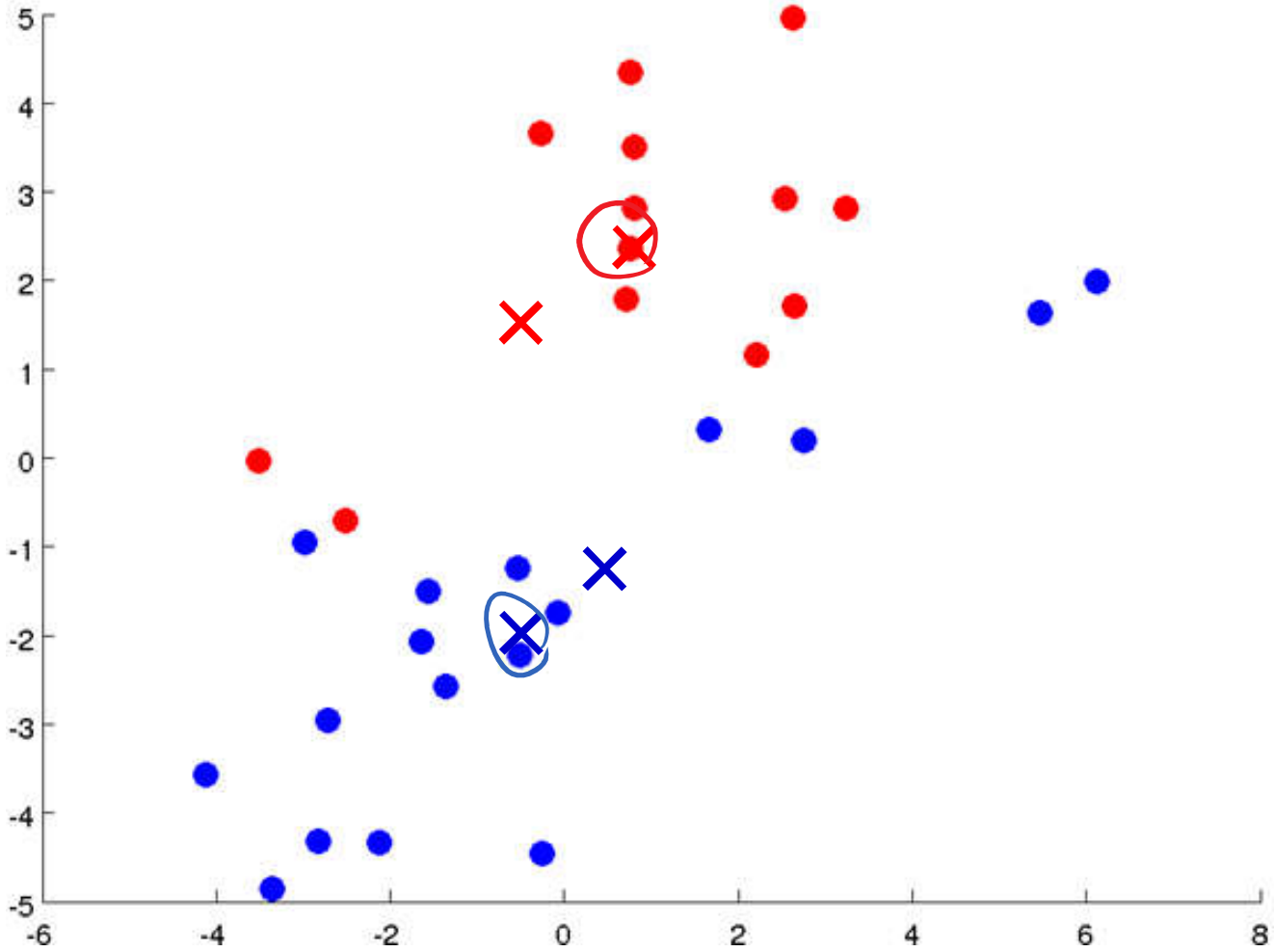

5
4
3
2
1
0
-1
-2
-3
-4
-5
-6
-4
-2
0
2
4
6
8

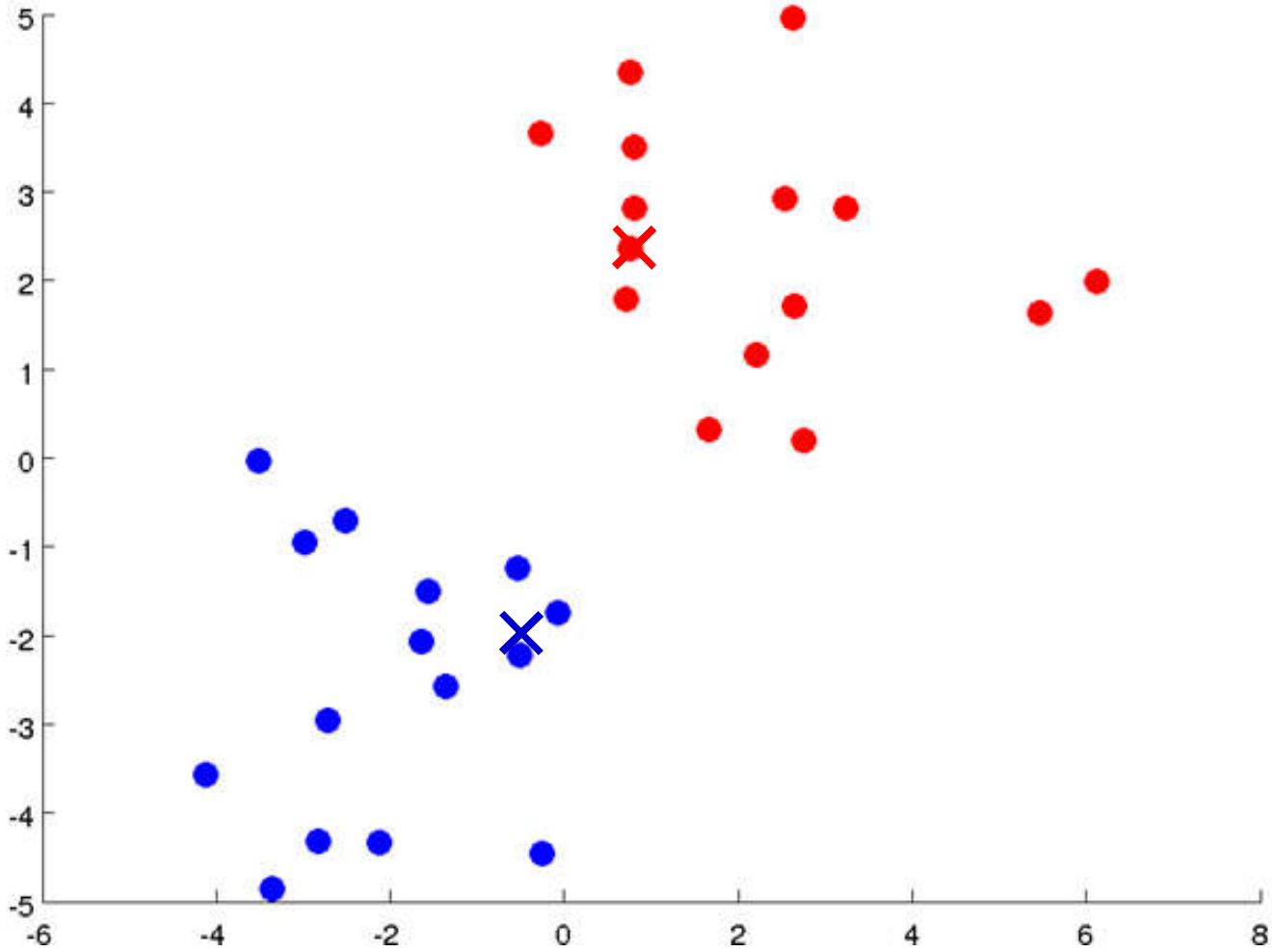

5
4
3
2
1
0
-1
-2
-3
-4
-5
-6
-4
-2
0
2
4
6
8
Cluster centroids

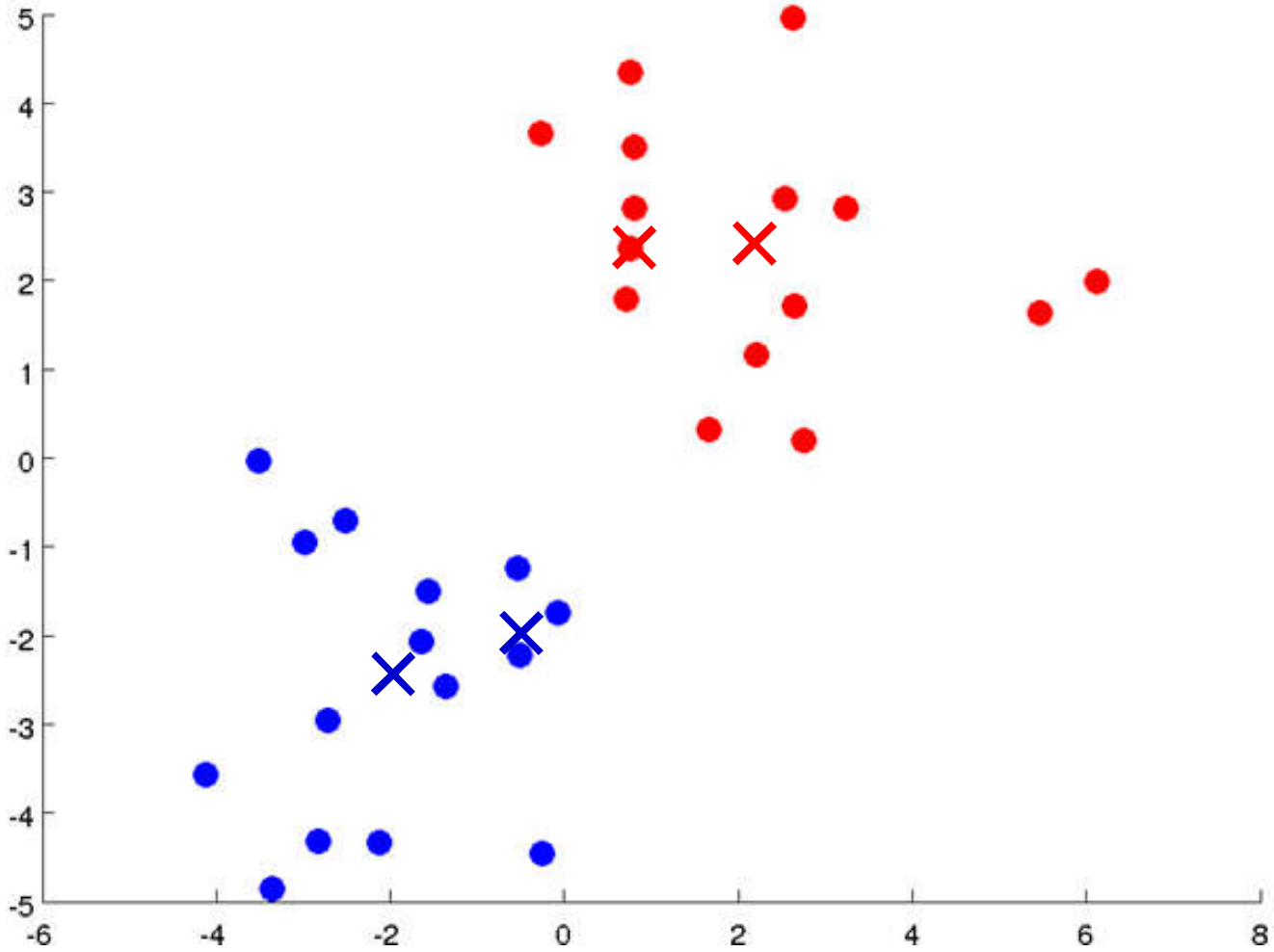

5
4
3
2
1
0
-1
-2
-3
-4
-5
-6
-4
-2
0
2
4
6
8

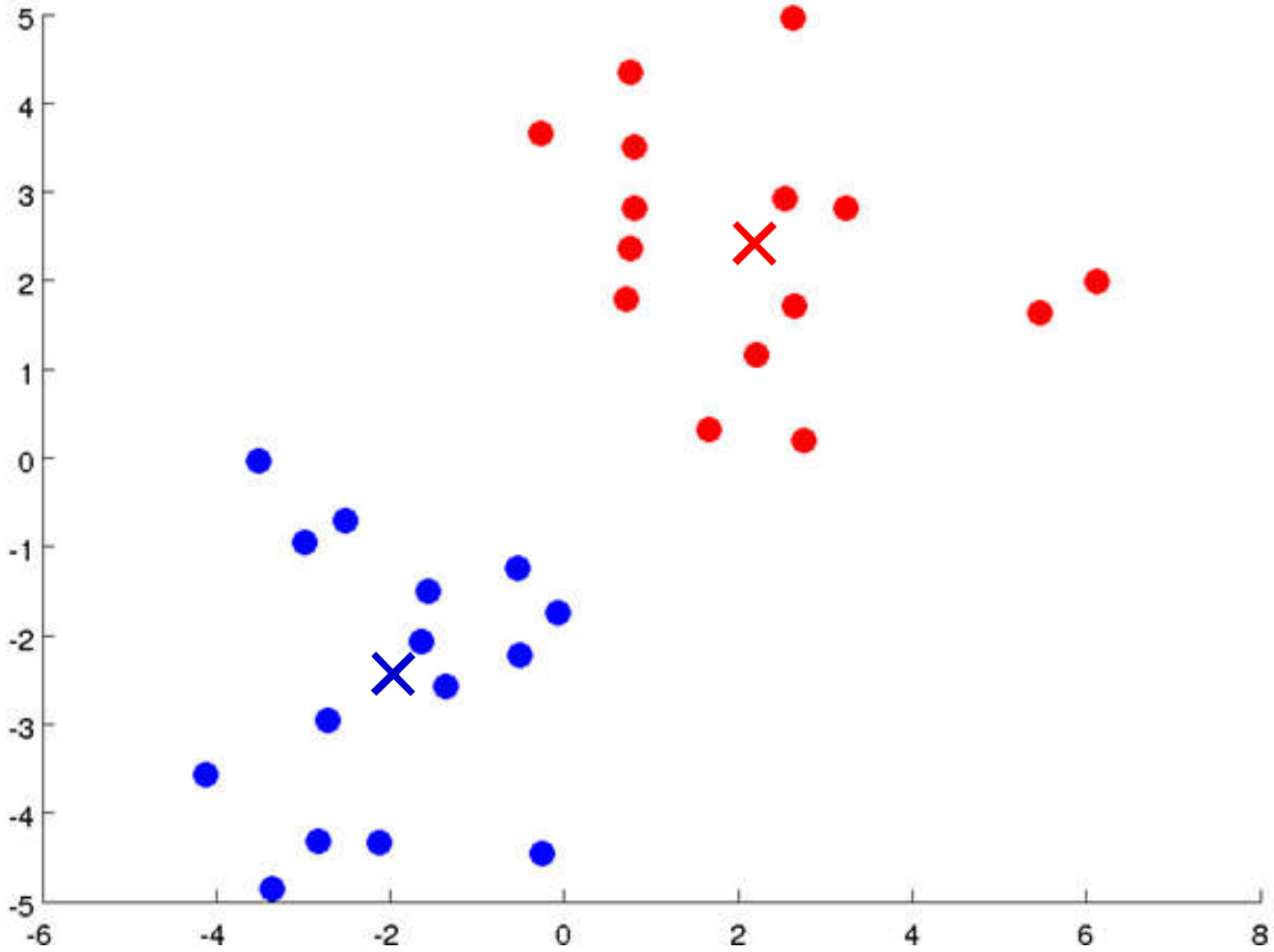

x_2
(x_1, x_2)
x_1
5
4
3
2
1
0
-1
-2
-3
-4
-5
-6
-4
-2
0
2
4
6
8


5
4
3
2
1
0
-1
-2
-3
-4
-5
-6
-4
-2
0
2
4
6
8


5
4
3
2
1
0
-1
-2
-3
-4
-5
-6
-4
-2
0
2
4
6
8


5
4
3
2
1
0
-1
-2
-3
-4
-5
-6
-4
-2
0
2
4
6
8


5
4
3
2
1
0
-1
-2
-3
-4
-5
-6
-4
-2
0
2
4
6
8


5
4
3
2
1
0
-1
-2
-3
-4
-5
-6
-4
-2
0
2
4
6
8

K-means algorithm

assumption

Input:

- K (number of clusters)
- Training set $\{x^{(1)}, x^{(2)}, \ldots, x^{(m)}\}$

$x^{(i)} \in \mathbb{R}^n$ (drop $x_0 = 1$ convention)

K-means algorithm

Randomly initialize K cluster centroids $\mu_1, \mu_2, \ldots, \mu_K \in \mathbb{R}^n$

Repeat {

for i = 1 to m

$c^{(i)}$:= index (from 1 to K) of cluster centroid closest to $x^{(i)}$

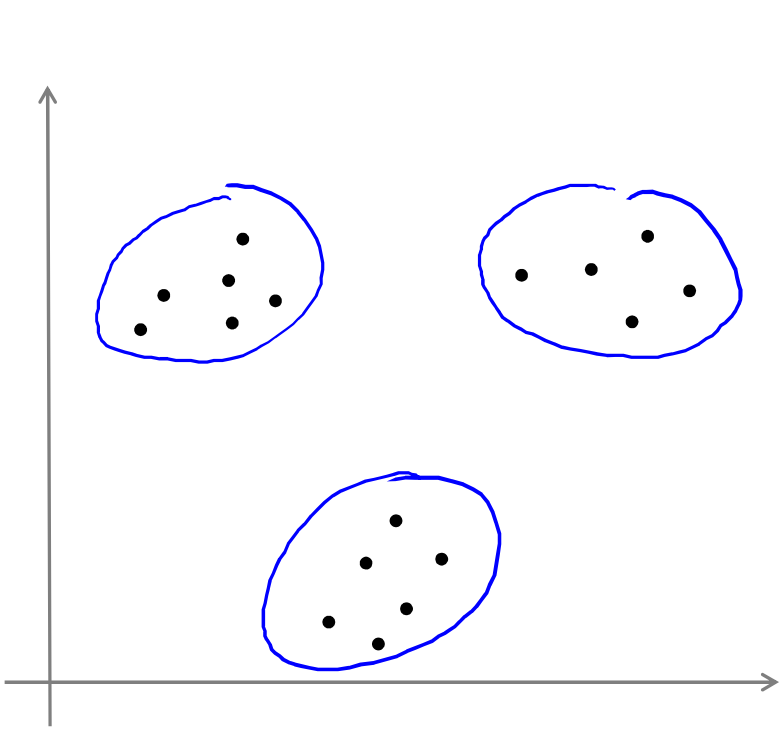
for k = 1 to K

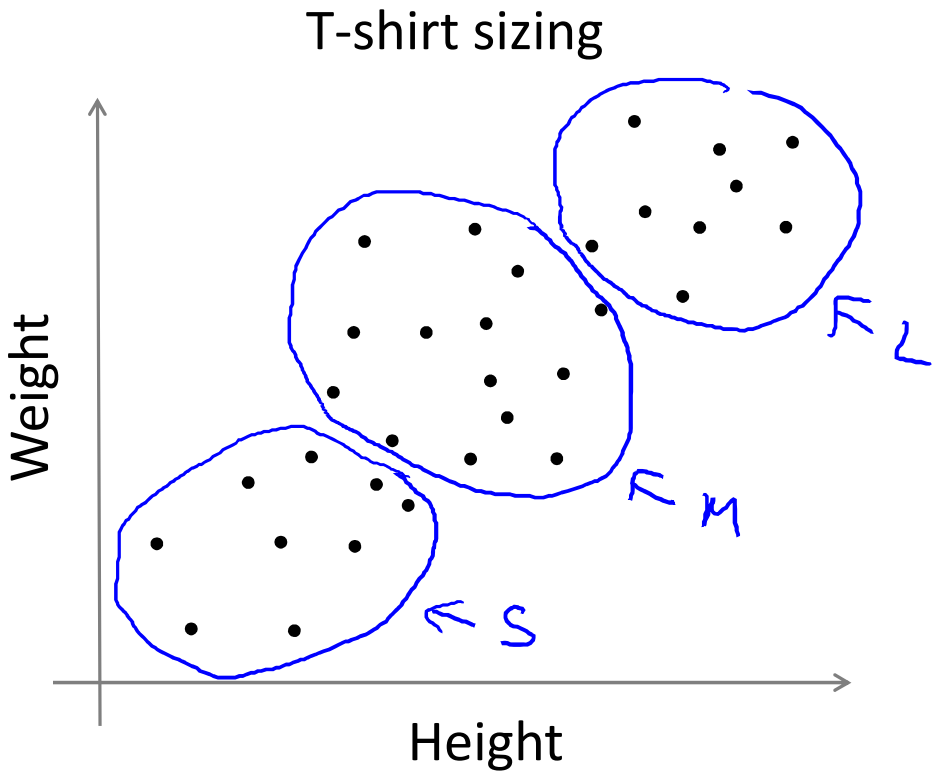
μ_k := average (mean) of points assigned to cluster k

}

K-means for non-separated clusters

S, M, L





Optimization objective

Optimization objective

K-means optimization objective

$c^{(i)}$ = index of cluster (1,2,…,K) to which example $x^{(i)}$ is currently assigned

μ_k = cluster centroid k ($\mu_k \in \mathbb{R}^n$)

$\mu_{c^{(i)}}$ = cluster centroid of cluster to which example $x^{(i)}$ has been assigned

Optimization objective:

$$J(c^{(1)}, \ldots, c^{(m)}, \mu_1, \ldots, \mu_K) = \frac{1}{m} \sum_{i=1}^{m} ||x^{(i)} - \mu_{c^{(i)}}||^2$$

$$\min_{\substack{c^{(1)},\ldots,c^{(m)},\\ \mu_1,\ldots,\mu_K}} J(c^{(1)}, \ldots, c^{(m)}, \mu_1, \ldots, \mu_K)$$

step 1 step 2

Random initialization

K-means algorithm

Randomly initialize K cluster centroids $\mu_1, \mu_2, \ldots, \mu_K \in \mathbb{R}^n$

Repeat {

for i = 1 to m

$c^{(i)}$:= index (from 1 to K) of cluster centroid closest to $x^{(i)}$

for k = 1 to K

μ_k := average (mean) of points assigned to cluster k

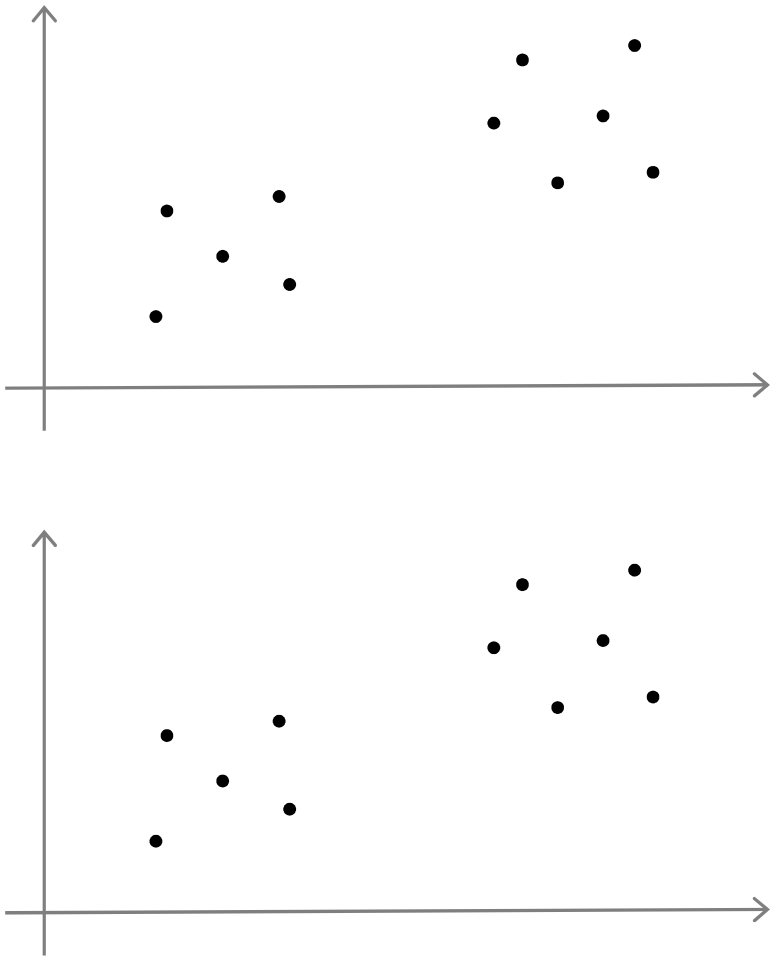
}

Random initialization

Should have $K < m$

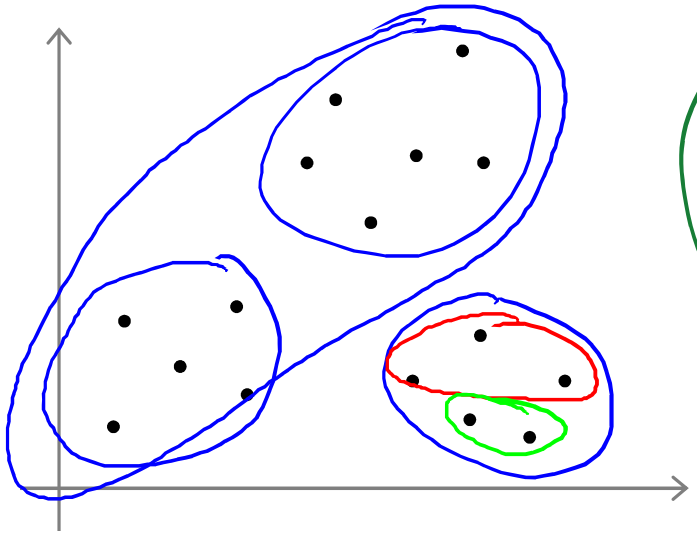
Randomly pick K training examples.

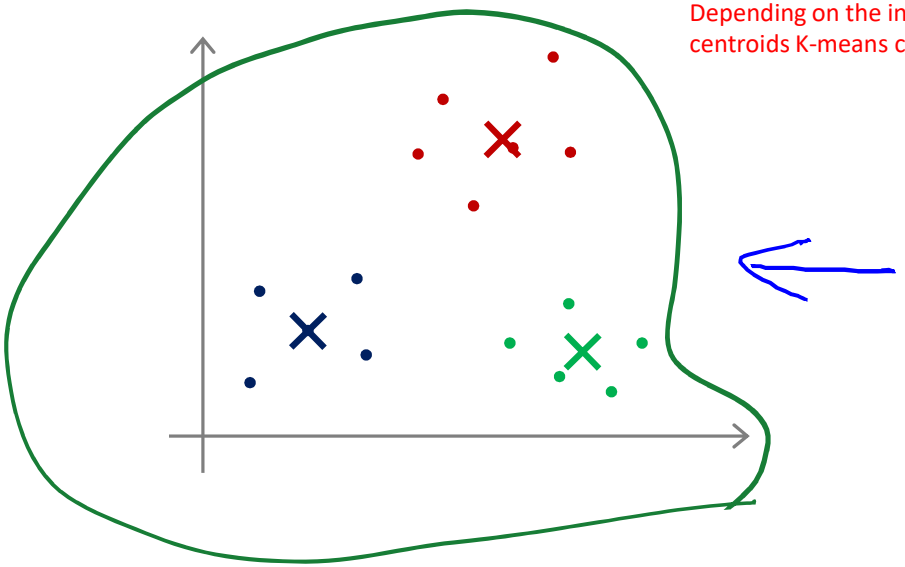
Set $\mu_1, \ldots, \mu_K$ equal to these K examples.



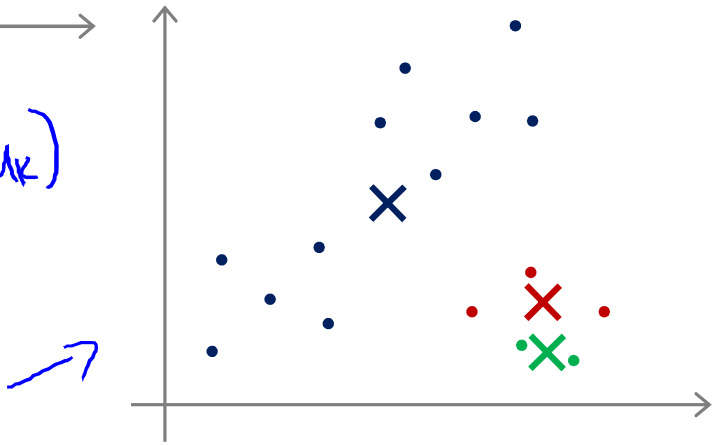
Local optima

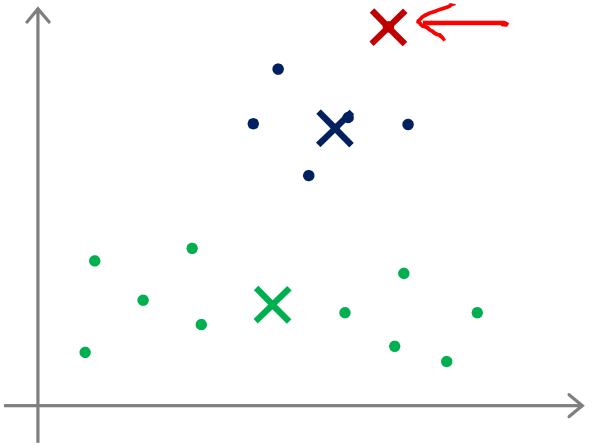
Depending on the initialization of cluster centroids K-means can produce different results





$J(c^{(1)}, \ldots, c^{(m)}, \mu_1, \ldots, \mu_K)$





Random initialization

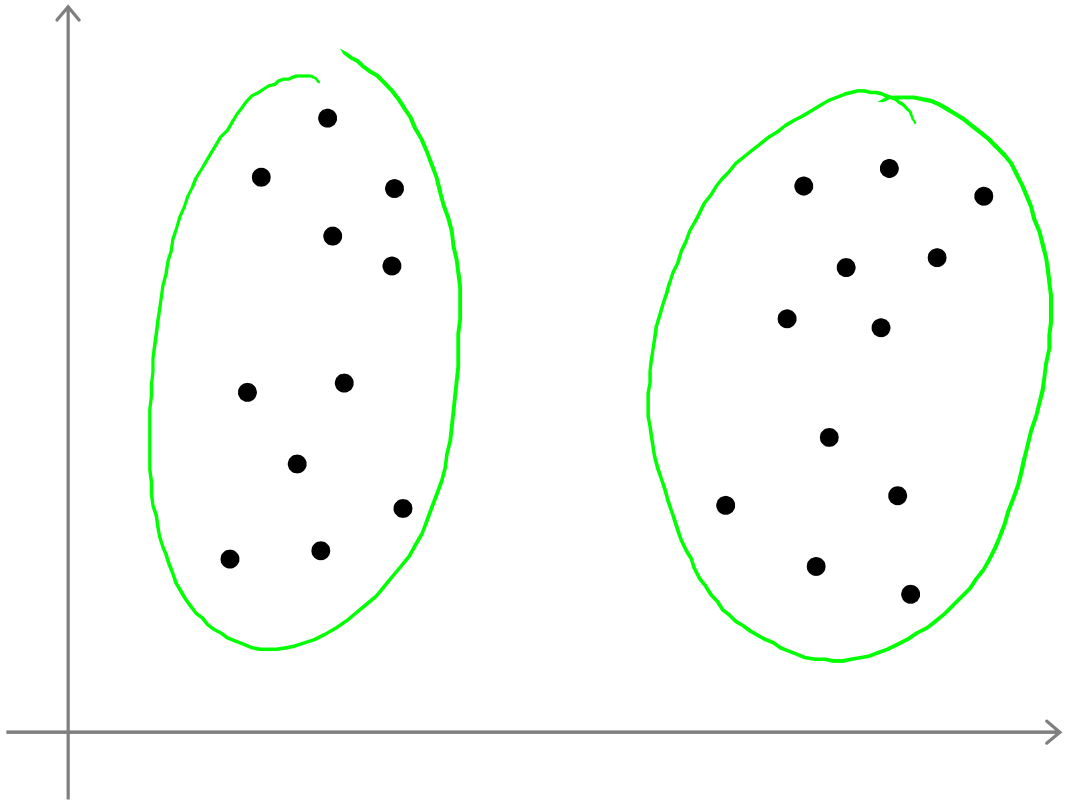
For i = 1 to 100 {

Randomly initialize K-means.
Run K-means. Get $c^{(1)}, \ldots, c^{(m)}, \mu_1, \ldots, \mu_K$
Compute cost function (distortion)
$J(c^{(1)}, \ldots, c^{(m)}, \mu_1, \ldots, \mu_K)$
}

Pick clustering that gave lowest cost $J(c^{(1)}, \ldots, c^{(m)}, \mu_1, \ldots, \mu_K)$

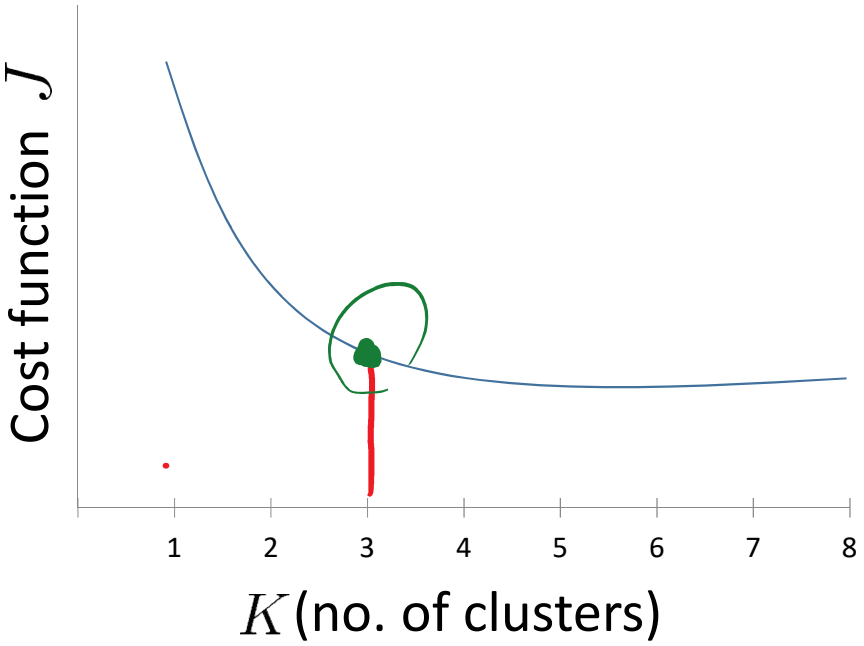
Choosing the number of clusters

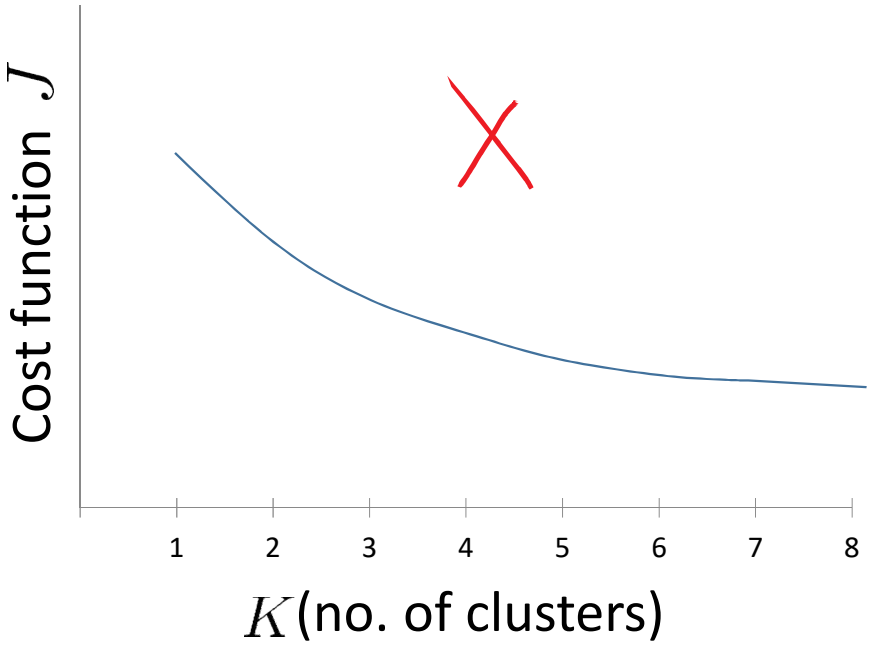
What is the right value of K?



Choosing the value of K

Elbow method:





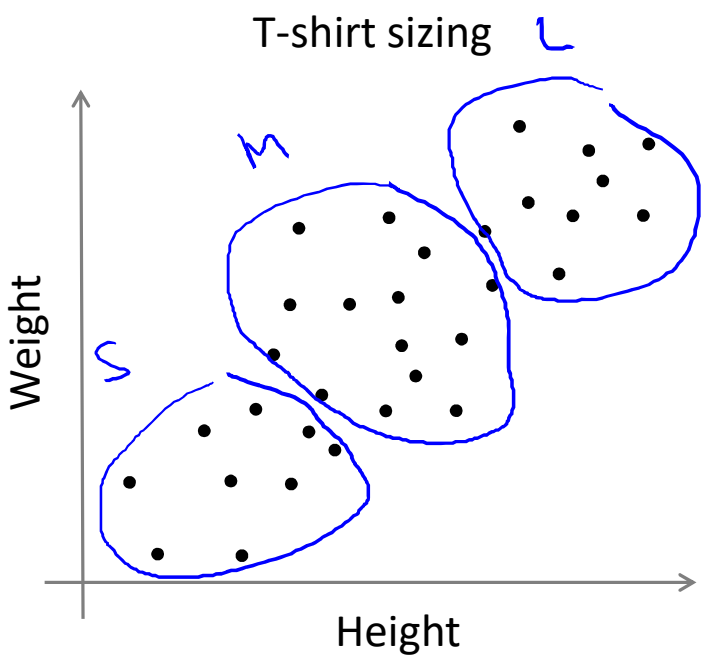
Choosing the value of K

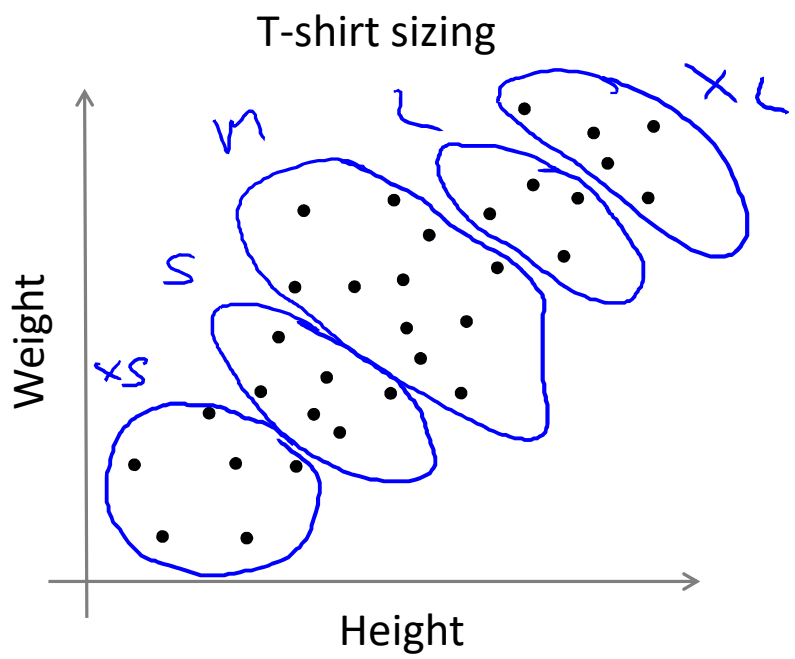
Sometimes, you're running K-means to get clusters to use for some later/downstream purpose. Evaluate K-means based on a metric for how well it performs for that later purpose.

K=3 S, M, L

K=5 XS, S, M, L, XL

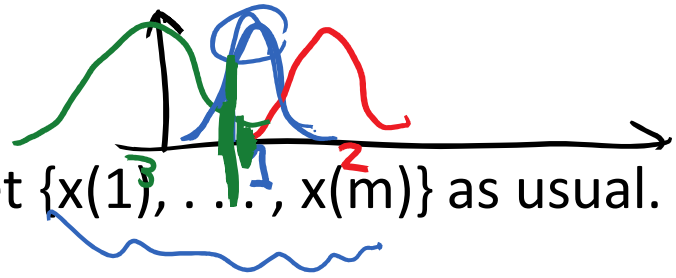
E.g.





Mixtures of Gaussians and the EM algorithm

Mixture of Gaussians model



- Suppose that we are given a training set {x(1), . . . , x(m)} as usual.
- We wish to model the data by specifying a joint distribution

$$p\left(x^{(i)}, z^{(i)}\right) = p\left(x^{(i)} \mid z^{(i)}\right) p\left(z^{(i)}\right).$$

- Here, $z^{(i)} \sim Categorical\ (\phi)$ (where $\phi_j \geq 0, \sum_{j=1}^{k} \phi_j = 1$, and the parameter ϕ_j gives $p\left(z^{(i)} = j\right)$), and $x^{(i)} \mid z^{(i)} = j \sim \mathcal{N}\left(\mu_j, \Sigma_j\right)$.
- We let k denote the number of values that the $z^{(i)}$'s can take on.
- $z^{(i)}$'s are latent random variables, meaning that they're hidden/unobserved.

Mixture of Gaussians model

- The parameters of our model are thus ϕ, μ and Σ. To estimate them, we can write down the likelihood of our data:

$$\ell(\phi, \mu, \Sigma) = \sum_{i=1}^{m} \log p\left(x^{(i)}; \phi, \mu, \Sigma\right)$$

$$= \sum_{i=1}^{m} \log \sum_{z^{(i)}=1}^{k} p\left(x^{(i)} \mid z^{(i)}; \mu, \Sigma\right) p\left(z^{(i)}; \phi\right)$$

The random variables $z^{(i)}$ indicate which of the k Gaussians each $x^{(i)}$ had come from. **Note that if we knew what the $z^{(i)}$'s were, the maximum likelihood problem would have been easy.** شبيه حل GDA

Mixture of Gaussians model

we could then write down the likelihood as

$$\ell(\phi, \mu, \Sigma) = \sum_{i=1}^{m} \log p\left(x^{(i)} \mid z^{(i)}; \mu, \Sigma\right) + \log p\left(z^{(i)}; \phi\right).$$

Maximizing this with respect to ϕ, μ and Σ gives the parameters:

$$\begin{cases} \phi_j & = \frac{1}{m}\sum_{i=1}^{m} 1\{z^{(i)} = j\}, \\ \mu_j & = \frac{\sum_{i=1}^{m} 1\{z^{(i)} = j\} x^{(i)}}{\sum_{i=1}^{m} 1\{z^{(i)} = j\}}, \\ \Sigma_j & = \frac{\sum_{i=1}^{m} 1\{z^{(i)} = j\}(x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^{m} 1\{z^{(i)} = j\}}. \end{cases}$$

Mixture of Gaussians model (EM algorithm)

- In our density estimation problem, the $z^{(i)}$'s are not known. What can we do?

The EM algorithm is an iterative algorithm that has two main steps.

Initialize μ, Σ, ϕ randomly

1. E-step, it tries to "guess" the values of the $z^{(i)}$'s.

2. M-step, it updates the parameters of our model based on our guesses.

μ, Σ, ϕ

EM algorithm

Here's the algorithm:

Repeat until convergence: {

(E-step) For each i, j, set

$$w_j^{(i)} := p\left(z^{(i)} = j \mid x^{(i)}; \phi, \mu, \Sigma\right)$$

(M-step) Update the parameters:

$$\phi_j := \frac{1}{m}\sum_{i=1}^{m} w_j^{(i)},$$

$$\mu_j := \frac{\sum_{i=1}^{m} w_j^{(i)} x^{(i)}}{\sum_{i=1}^{m} w_j^{(i)}},$$

$$\Sigma_j := \frac{\sum_{i=1}^{m} w_j^{(i)} \left(x^{(i)} - \mu_j\right)\left(x^{(i)} - \mu_j\right)^T}{\sum_{i=1}^{m} w_j^{(i)}} \}$$

EM algorithm

In the E-step, we calculate the posterior probability of our parameters the $z^{(i)}$'s, given the $x^{(i)}$ and using the current setting of our parameters.

$$p\left(z^{(i)} = j \mid x^{(i)}; \phi, \mu, \Sigma\right) = \frac{p\left(x^{(i)} \mid z^{(i)} = j; \mu, \Sigma\right) p\left(z^{(i)} = j; \phi\right)}{\sum_{l=1}^{k} p(x^{(i)} \mid z^{(i)} = l; \mu, \Sigma) p(z^{(i)} = l; \phi)}$$

Here, $p\left(x^{(i)} \mid z^{(i)} = j; \mu, \Sigma\right)$ is given by evaluating the density of a Gaussian with mean μ_j and covariance Σ_j at $x^{(i)}$; $p\left(z^{(i)} = j; \phi\right)$ is given by ϕ_j, and so on.

Mixture of Gaussians model

- The EM-algorithm is also reminiscent of the K-means clustering algorithm, except that instead of the "hard" cluster assignments $c(i)$, we instead have the "soft" assignments $w_j^{(i)}$.
- Similar to K-means, it is also susceptible to local optima, so reinitializing at several different initial parameters may be a good idea.